

UNIVERSIDAD DE SALAMANCA

GRADO EN INGENIERIA INFORMATICA

ANEXO IV

Documentacion Tecnica

Sistema de Gestion Documental mediante Blockchain



David Pérez Velasco

Tutores:

Gabriel Villarrubia Gonzalez

Sergio Garcia Gonzalez

Julio 2026

Trabajo de Fin de Grado

Índice

1. Introduccion	4
2. Tecnologias y lenguajes	4
2.1. Frontend	4
2.2. Backend	5
2.3. Blockchain	6
2.4. Infraestructura	7
3. Marcos de trabajo y bibliotecas	8
3.1. Frontend	8
3.2. Backend	9
3.3. Smart contracts	9
4. Estructura del codigo	9
4.1. Arbol de directorios general	10
4.2. Frontend	10
4.3. Backend	10
4.4. Smart contracts	11
4.5. Infraestructura	11
5. Metodos principales	11
5.1. Cifrado de documentos	11
5.2. Flujo prepare/confirm	12
5.3. Sincronizacion blockchain	13
6. Smart Contracts	14
6.1. DocumentRegistry.sol	14
6.2. Eventos on-chain y verificacion funcional	14
6.3. Decision de consolidacion en un unico contrato	14
6.4. Decisiones de diseno del contrato consolidado	14
7. Arquitectura de cifrado	15
7.1. Capa 1: Autenticacion	15
7.2. Capa 2: Cifrado de contenido	15
7.3. Capa 3: Blockchain	16
8. Integracion de capas de persistencia	16
8.1. Funcionamiento de IPFS	17
8.2. Flujo de lectura/escritura coordinado	17
9. Smart contract consolidado DocumentRegistry	18
9.1. Estructuras de datos on-chain	18
9.2. Operaciones registradas en cadena	18
9.3. Eventos y restricciones de seguridad	19
10. Patron prepare/confirm y sincronizacion	20
11. Observabilidad blockchain y sincronizacion de estado	20
11.1. El patron de observabilidad mediante Event Listener	20
11.2. Flujo completo de una operacion documental	21

12.Gestion de contraseñas, frases de recuperacion y claves	21
12.1. Derivacion de claves con Argon2	21
12.2. Cifrado de claves privadas con frase de recuperacion	22
13.Despliegue tecnico con Docker	22
13.1. Arquitectura de contenedores	22
13.2. Servicios del docker-compose	22
13.3. Volúmenes, redes y variables de entorno	23
13.4. Integracion continua (CI/CD)	23
14.Glosario tecnico	24
15.Conclusiones del Anexo IV	25

1. Introduccion

Este anexo es la documentacion tecnica del sistema. Esta pensado para que un desarrollador que no conozca el proyecto pueda entender como esta montado, donde esta cada cosa y por que se tomaron ciertas decisiones. No es un documento para leerse de principio a fin, sino mas bien un mapa: cada seccion explica una parte del sistema con el nivel de detalle suficiente para que alguien pueda modificarlo sin romper nada.

Las siguientes secciones cubren las tecnologias que se usan, como esta organizado el codigo, los metodos mas importantes del backend y del frontend, los contratos inteligentes, como funciona el cifrado y como se despliega todo con Docker. Al final hay un glosario con los terminos tecnicos que aparecen a lo largo del texto. Se recomienda tener el codigo fuente a mano mientras se lee este anexo.

2. Tecnologias y lenguajes

El stack tecnologico de DocumentChain se ha elegido buscando sobre todo seguridad, tipado estatico y que el entorno se pueda reproducir facilmente. A continuacion se detallan las tecnologias de cada capa.

2.1. Frontend

La capa de presentacion es una aplicacion de pagina unica (SPA, *Single Page Application*) que se comunica con el backend mediante una API REST y con la blockchain mediante librerias especializadas, por lo que la arquitectura resultante separa claramente las responsabilidades de cada capa. El conjunto de tecnologias se resume en la Tabla 1.

Tecnologia	Version	Proposito y justificacion
React	19	Biblioteca declarativa para la construccion de interfaces de usuario basadas en componentes reutilizables. La version 19 mejora el rendimiento del renderizado concurrente y simplifica la gestion de referencias.
TypeScript	5.9	Superconjunto tipado de JavaScript que detecta errores en tiempo de compilacion, reduce la deuda tecnica y facilita la refactorizacion segura en un proyecto de estas dimensiones.
Vite	7	Herramienta de construccion que proporciona un entorno de desarrollo rapido gracias al Hot Module Replacement (HMR) y genera bundles optimizados para produccion mediante Rollup.
Tailwind CSS	3	Framework de utilidades CSS que permite disenar interfaces responsive y consistentes sin salir del marcado HTML, reduciendo la cantidad de codigo CSS ad hoc.
shadcn/ui	–	Coleccion de componentes accesibles y personalizables contruidos sobre Radix UI y Tailwind CSS, que acelera el desarrollo de formularios, dialogos y tablas complejas.
Zustand	5	Libreria ligera de gestion de estado global que evita la complejidad de Redux manteniendo un store tipado y reactivo.
React Query (TanStack)	5	Gestor de estado asincrono y cacheo de datos de API que simplifica la sincronizacion entre servidor y cliente, incluyendo reintentos automaticos y paginacion.
Axios	–	Cliente HTTP configurado con interceptores para inyeccion de tokens JWT y manejo centralizado de errores de red.
Ethers.js	6.x	Libreria para la interaccion con wallets Ethereum y smart contracts, compatible con los estandares EIP-6963 y WalletConnect.
Socket.io-client	–	Cliente de comunicacion en tiempo real para recibir notificaciones push del backend sin necesidad de polling.

Cuadro 1: Tecnologias principales del frontend

React 19 con TypeScript se eligio para mantener un codigo frontend predecible y escalable. Vite reduce los tiempos de compilacion respecto a herramientas tradicionales, lo que se nota bastante durante la fase de desarrollo activo. Tailwind CSS y shadcn/ui garantizan una interfaz coherente con el Manual de Identidad Corporativa de la Universidad de Salamanca sin requerir un equipo de disenio dedicado.

2.2. Backend

El backend es una API REST desarrollada sobre Node.js que centraliza la logica de negocio, el acceso a la base de datos, la interaccion con IPFS y la sincronizacion con la blockchain. La

Tabla 2 recoge las tecnologias fundamentales.

Tecnologia	Version	Proposito y justificacion
Node.js	20 LTS	Entorno de ejecucion JavaScript del lado del servidor, seleccionado por su modelo de eventos no bloqueante y su amplio ecosistema de paquetes.
Express	5	Framework web minimalista que gestiona rutas, middleware y peticiones HTTP con un overhead reducido.
TypeScript	5.9	Tipado estatico compartido con el frontend, que facilita la coherencia de modelos de datos entre ambas capas.
Prisma	5.22	ORM de siguiente generacion que proporciona tipado seguro, migraciones versionadas y un cliente auto-generado a partir del esquema de base de datos.
PostgreSQL	16	Sistema gestor de bases de datos relacionales objeto, elegido por su robustez, soporte para tipos JSONB y extensiones espaciales.
Argon2	–	Algoritmo moderno de derivacion de claves para el hash seguro de contraseñas, ganador del Password Hashing Competition.
JWT (jsonwebtoken)	–	Generacion y validacion de tokens firmados para la gestion de sesiones sin estado.
Multer	–	Middleware para la recepcion de archivos multipart en las rutas de subida de documentos.
Winston	–	Sistema de logging estructurado con transportes configurables para consola y ficheros rotativos.
Node-cron	–	Programador de tareas que ejecuta el worker de sincronizacion blockchain cada cinco minutos.
Nodemailer	–	Envio de correos electronicos transaccionales (verificacion de cuenta, recuperacion de acceso).
Socket.io	–	Servidor WebSocket para la emision de eventos en tiempo real hacia los clientes conectados.
Jest	–	Framework de testing para el backend, combinado con Supertest para pruebas de integracion sobre endpoints HTTP.

Cuadro 2: Tecnologias principales del backend

La combinacion de Express y TypeScript permite construir una API tipada y modular, mientras que Prisma elimina la necesidad de escribir consultas SQL manuales para la mayoria de operaciones, reduciendo el riesgo de inyeccion y errores de tipado. Argon2 sustituye a bcrypt como algoritmo de hash de contraseñas por su mayor resistencia a ataques de paralelizacion en GPU.

2.3. Blockchain

La capa blockchain proporciona inmutabilidad y trazabilidad a los eventos criticos del sistema. Las tecnologias especificas se listan en la Tabla 3.

Tecnologia	Version	Proposito y justificacion
Solidity	0.8.20	Lenguaje de programacion para smart contracts en la Ethereum Virtual Machine (EVM), con mejoras en seguridad y optimizacion de gas.
Hardhat	2.22	Entorno de desarrollo, testing y despliegue de smart contracts que incluye una red local, tareas personalizadas y plugins de cobertura.
OpenZeppelin Contracts	5.x	Libreria de referencia para patrones seguros de smart contracts, incluyendo control de acceso y proteccion contra reentrancia.
Ethers.js	6.x	Libreria TypeScript para la interaccion con nodos Ethereum, wallets y contratos inteligentes desde el frontend y el backend.

Cuadro 3: Tecnologias principales de la capa blockchain

La version 0.8.20 de Solidity se ha seleccionado por su compatibilidad con los ultimos patrones de OpenZeppelin y su soporte para operaciones de verificacion personalizada. Hardhat permite ejecutar tests automatizados en una red local antes de cualquier despliegue, lo que reduce drasticamente el riesgo de errores en produccion.

2.4. Infraestructura

La infraestructura de despliegue se basa en contenedores Docker y servicios auxiliares que garantizan la reproducibilidad del entorno. La Tabla 4 detalla los componentes.

Tecnologia	Version	Proposito y justificacion
Docker	–	Plataforma de contenerizacion que empaqueta cada servicio con sus dependencias, eliminando los problemas de “funciona en mi maquina”.
Docker Compose	–	Orquestacion declarativa de multi-contenedores que permite levantar toda la arquitectura con un unico comando.
Nginx	–	Servidor proxy inverso y balanceador de carga que sirve el frontend estatico, termina conexiones SSL/TLS y redirige peticiones API al backend.
GitHub Actions	–	Plataforma de integracion y entrega continuas (CI/CD) que automatiza la ejecucion de tests, la construccion de imagenes y el despliegue en servidores autoalojados.

Cuadro 4: Tecnologias principales de infraestructura

El uso de Docker Compose como orquestador simplifica la gestion de redes virtuales, volúmenes persistentes y variables de entorno, permitiendo clonar el repositorio y disponer de un entorno funcional en cuestion de minutos.

3. Marcos de trabajo y bibliotecas

Ademas de las tecnologias base, el proyecto integra multiples bibliotecas especializadas que aceleran el desarrollo y mejoran la experiencia del usuario final.

3.1. Frontend

El frontend se apoya en un ecosistema de componentes y utilidades que se detallan en la Tabla 5.

Biblioteca	Version	Proposito
react-router-dom	7	Enrutamiento declarativo con soporte para carga diferida (lazy loading) y proteccion de rutas mediante guards.
@tanstack/react-query	5	Gestion de estado asincrono, cacheo inteligente de peticiones y sincronizacion de fondo (background refetching).
zustand	5	Gestion de estado global ligera para datos que no provienen del servidor, como preferencias de interfaz y estados de modales.
react-hook-form	7	Gestion de formularios con validacion reactiva que minimiza los renderizados innecesarios.
zod	4	Declaracion y validacion de esquemas TypeScript compartidos entre frontend y backend.
axios	–	Cliente HTTP con interceptores para inyeccion de tokens y manejo centralizado de errores.
socket.io-client	–	Comunicacion bidireccional en tiempo real con el servidor para notificaciones push.
ethers	v6	Interaccion con wallets (MetaMask, WalletConnect) y ejecucion de llamadas a smart contracts.
@walletconnect/ethereum-provider	–	Conexion con wallets moviles mediante el protocolo WalletConnect v2.
lucide-react	–	Iconos vectoriales de linea fina con estilo consistente y personalizable via Tailwind CSS.
recharts	–	Visualizacion de datos estadisticos en paneles administrativos mediante graficos interactivos.

Cuadro 5: Bibliotecas principales del frontend

La combinacion de react-hook-form y zod permite definir los esquemas de validacion una unica vez en TypeScript y reutilizarlos tanto en el cliente como en el servidor, garantizando la coherencia de restricciones.

3.2. Backend

El backend incorpora bibliotecas especializadas para seguridad, comunicaciones y tareas programadas, como se recoge en la Tabla 6.

Biblioteca	Version	Proposito
jsonwebtoken	–	Generacion y verificacion de tokens de acceso (15 min) y refresh (7 dias).
bcrypt/argon2	–	Derivacion segura de contraseñas mediante Argon2id (recomendado) con parametros de memoria y paralelismo configurables.
multer	–	Recepcion de ficheros multipart con filtros de tipo MIME y limites de tamaño.
nodemailer	–	Envio de correos electronicos transaccionales mediante SMTP o transportes de prueba.
winston	–	Registro estructurado de logs con niveles (debug, info, warn, error) y rotacion de ficheros.
node-cron	–	Ejecucion periodica del worker de sincronizacion blockchain cada cinco minutos.
socket.io	–	Emision de eventos en tiempo real a salas (rooms) organizadas por identificador de usuario.
ethers	v6	Lectura de eventos on-chain, preparacion de transacciones y verificacion de firmas de mensajes mediante wallet.

Cuadro 6: Bibliotecas principales del backend

3.3. Smart contracts

Los contratos inteligentes se construyen sobre las utilidades de OpenZeppelin, que proporcionan implementaciones auditadas de patrones de seguridad. La Tabla 7 enumera las dependencias principales.

Dependencia	Proposito
@openzeppelin/contracts	Libreria de contratos estandar: AccessControl, ReentrancyGuard, EnumerableSet.
hardhat-docgen	Plugin para la generacion automatica de documentacion NatSpec en formato HTML.
solidity-coverage	Instrumentacion de cobertura de codigo para tests de contratos.

Cuadro 7: Dependencias principales de los smart contracts

4. Estructura del codigo

El repositorio sigue una organizacion monorepo que agrupa frontend, backend, smart contracts e infraestructura en un unico arbol versionado. Esta estructura facilita la coherencia de

tipos compartidos, la ejecucion conjunta de tests y la orquestacion mediante Docker Compose.

4.1. Arbol de directorios general

La raiz del proyecto contiene los directorios principales y los ficheros de configuracion compartida:

```
documentchain/
|-- frontend/           # Aplicacion SPA (React + Vite)
|-- backend/           # API REST (Node.js + Express)
|-- smart-contracts/    # Contratos Solidity y tests
|-- docker-compose.yml  # Orquestacion de servicios
|-- nginx/             # Configuracion del proxy inverso
|-- .github/           # Workflows de CI/CD
|-- README.md
```

4.2. Frontend

El codigo fuente del frontend se organiza en modulos funcionales que separan la presentacion de la logica de estado:

```
frontend/
|-- public/             # Assets estaticos (favicon, imagenes)
|-- src/
| |-- components/       # Componentes React reutilizables (UI)
| |-- pages/           # Vistas de alto nivel mapeadas a rutas
| |-- hooks/           # Custom hooks (useDebounce, useSocketListener)
| |-- api/             # Instancias axios y funciones de llamada a la API
| |-- types/           # Definiciones TypeScript compartidas
| |-- lib/             # Utilidades, helpers y formateadores
| |-- stores/          # Definiciones de Zustand para estado global
| |-- providers/       # Proveedores de contexto (Web3, tema, notificaciones)
| |-- App.tsx          # Punto de entrada y enrutamiento
| |-- main.tsx         # Montaje en el DOM y configuracion inicial
|-- package.json
|-- vite.config.ts
|-- tailwind.config.js
```

Cada directorio tiene una responsabilidad clara. El directorio **components/** aloja unicamente componentes de presentacion sin logica de negocio; **pages/** agrupa las vistas que consumen hooks y servicios; **api/** centraliza las peticiones HTTP para facilitar la inyeccion de tokens y el manejo de errores global.

4.3. Backend

El backend sigue una arquitectura por capas que separa controladores, servicios y utilidades:

```
backend/
|-- src/
| |-- controllers/      # Handlers HTTP de Express (req/res)
| |-- services/        # Logica de negocio pura
| |-- middleware/      # Middleware de auth, validacion y errores
| |-- lib/             # Utilidades (crypto, blockchain, ipfs)
| | |-- crypto/        # Cifrado AES-256-GCM, intercambio ECDH
| | |-- blockchain/    # Wrapper de ethers.js y event listeners
| | |-- ipfs/          # Cliente del nodo Kubo
| |-- prisma/          # Esquema declarativo de la base de datos
| | |-- schema.prisma  # Historial de migraciones versionadas
| | |-- migrations/
| |-- routes/          # Definicion de rutas y asociacion a controladores
| |-- types/           # Tipos TypeScript especificos del backend
| |-- index.ts         # Punto de entrada y arranque del servidor
|-- tests/             # Tests de integracion con Jest y Supertest
|-- package.json
|-- tsconfig.json
```

La carpeta **services/** encapsula las reglas de negocio y es la unica autorizada para invocar al cliente de Prisma. Los **controllers/** se limitan a extraer datos de la peticion, invocar al servicio correspondiente y formatear la respuesta HTTP.

4.4. Smart contracts

Los contratos inteligentes se gestionan en un modulo independiente con su propio ciclo de compilacion y despliegue:

```
smart-contracts/
|-- contracts/
| |-- DocumentRegistry.sol      # Contrato consolidado principal
|-- test/                      # Tests unitarios en JavaScript/TypeScript
|-- scripts/
| |-- deploy.ts                # Script de despliegue automatizado
|-- hardhat.config.ts          # Configuracion de redes, compilador y plugins
|-- package.json
```

El directorio **test/** contiene escenarios de prueba que verifican no solo el comportamiento funcional, sino tambien las restricciones de seguridad (reentrancia, permisos).

4.5. Infraestructura

Los ficheros de infraestructura permiten reproducir el entorno completo en cualquier maquina con Docker:

```
documentchain/
|-- docker-compose.yml          # Definicion de servicios, redes y volúmenes
|-- nginx/
| |-- nginx.conf                # Configuracion del proxy inverso y SPA fallback
|-- .github/
| |-- workflows/
| | |-- ci.yml                  # Pipeline de integracion continua
|-- scripts/
| |-- reseed-dev.ps1           # Reseteo y resemilla del entorno de desarrollo
```

5. Metodos principales

Esta seccion documenta los algoritmos y flujos criticos del sistema, cuyo correcto funcionamiento garantiza la seguridad de los documentos y la consistencia entre la base de datos y la blockchain.

5.1. Cifrado de documentos

El sistema implementa un esquema de cifrado hibrido que combina la eficiencia del cifrado simetrico con la seguridad del intercambio de claves asimetrico. El flujo completo se describe a continuacion y se ilustra con el pseudocodigo de la List 1.

Fase 1: Generacion de clave simetrica. Para cada documento se genera una clave unica de 256 bits mediante un generador criptograficamente seguro (`crypto.randomBytes(32)`). Esta clave nunca se reutiliza entre documentos, de modo que el compromiso de un fichero no afecta al resto.

Fase 2: Cifrado del archivo. El fichero en bruto se cifra mediante AES-256-GCM (Galois/Counter Mode), que proporciona confidencialidad e integridad simultaneas. Se genera un vector de inicializacion (IV) aleatorio de 128 bits y se extrae el tag de autenticacion tras finalizar el cifrado.

Fase 3: Cifrado de la clave. La clave simetrica del documento se cifra con la clave publica del propietario mediante un esquema hibrido basado en ECDH (Elliptic Curve Diffie-Hellman)

sobre la curva P-256. El resultado es un encapsulado que únicamente el poseedor de la clave privada correspondiente puede descifrar.

Fase 4: Almacenamiento distribuido. El contenido cifrado (archivo + IV + tag) se sube al nodo IPFS, obteniéndose un CID único. La clave cifrada, el IV y el tag se almacenan en PostgreSQL asociados al registro del documento.

Listing 1: Pseudocódigo del flujo de cifrado completo

```
funcion cifrarDocumento(archivo: Buffer, clavePublica: Buffer):
  // 1. Clave simetrica unica
  claveAES = generarBytesAleatorios(32)

  // 2. Cifrado del contenido
  iv = generarBytesAleatorios(16)
  cifrado = AES-256-GCM(archivo, claveAES, iv)

  // 3. Cifrado de la clave con clave publica del propietario
  claveCifrada = ECDH-encapsular(clavePublica, claveAES)

  // 4. Persistencia
  cid = ipfs.agregar(cifrado.ciphertext || iv || cifrado.tag)
  baseDatos.guardar({
    documentId: uuid(),
    cid: cid,
    encryptedKey: claveCifrada,
    iv: iv,
    authTag: cifrado.tag
  })
  retornar cid
fin funcion
```

El método `encryptFile` de `FileCrypto.ts` implementa directamente las fases 1 y 2, mientras que `KeyManager.ts` gestiona las fases 3 y 4, manteniendo una separación de responsabilidades que facilita la auditoría.

5.2. Flujo prepare/confirm

Las operaciones que modifican el estado documental en la blockchain requieren la firma de la transacción con la clave privada del usuario, que nunca abandona su wallet. Para coordinar esta firma sin bloquear la interfaz, se diseñó un flujo de cuatro fases: preparación, firma, confirmación y sincronización.

Fase 1: Preparación. El backend valida permisos, cifra el documento si es privado, lo sube a IPFS y crea un registro en PostgreSQL con estado `PREPARING`. Devuelve al frontend los datos necesarios para construir la transacción (`to`, `data`, `value`).

Fase 2: Firma. El frontend utiliza `Ethers.js` para construir y firmar la transacción con la wallet del usuario. La transacción se envía a la red y se obtiene el *transaction hash*.

Fase 3: Confirmación. El frontend envía el *transaction hash* al backend, que actualiza el registro a estado `TX_SUBMITTED` y almacena el hash como evidencia de envío.

Fase 4: Sincronización. Un worker periódico consulta el estado de las transacciones pendientes. Si el *receipt* indica éxito, el registro pasa a `SYNCED`; si fue revertida, pasa a `FAILED`.

Listing 2: Pseudocódigo del flujo prepare/confirm

```
funcion prepararDocumento(hashContenido, cid, usuario):
  validarPermisos(usuario)
  si documento.esPrivado entonces
    cid = cifrarYSubirAIPFS(documento)
  fin si
  registro = baseDatos.crear({
    hash: hashContenido,
    cid: cid,
    estado: 'PREPARING',
    usuarioId: usuario.id
  })
```

```

    })
    txData = contrato.populateTransaction('createDocument', hash, cid)
    retornar { registroId: registro.id, txData: txData }
  fin funcion

  funcion confirmarDocumento(registroId, txHash):
    registro = baseDatos.obtener(registroId)
    registro.txHash = txHash
    registro.estado = 'TX_SUBMITTED'
    baseDatos.guardar(registro)
  fin funcion

  funcion sincronizarWorker():
    pendientes = baseDatos.buscarPorEstado('TX_SUBMITTED')
    para cada p en pendientes:
      receipt = proveedor.obtenerReceipt(p.txHash)
      si receipt.status == 1 entonces
        p.estado = 'SYNCED'
        p.blockNumber = receipt.blockNumber
        notificar(p.usuarioId, 'Documento sincronizado')
      sino
        p.estado = 'FAILED'
        notificar(p.usuarioId, 'Transaccion revertida')
      fin si
      baseDatos.guardar(p)
    fin para
  fin funcion

```

Este patron desacopla la logica de negocio del backend de la interaccion directa con la wallet, permitiendo soportar multiples proveedores (MetaMask, WalletConnect, Coinbase Wallet) sin modificar la API REST.

5.3. Sincronizacion blockchain

La consistencia entre la base de datos relacional y la blockchain se mantiene mediante dos mecanismos complementarios: un listener de eventos en tiempo real y un worker de reconciliacion periodica.

Event Listener. Al iniciarse, el backend suscribe una instancia del contrato a los eventos reales del contrato DocumentRegistry: DocumentCreated, VersionCreated, VersionRestored, DocumentSigned, DocumentShared, PermissionRevoked, OwnershipTransferred, DocumentArchived, DocumentDeleted y OperationalVersionChanged. Cuando se emite un evento, el callback actualiza el estado correspondiente en PostgreSQL y emite un mensaje WebSocket a los clientes conectados.

Worker de sincronizacion. Ejecutado cada cinco minutos mediante `node-cron`, este worker consulta los registros en estado `TX_SUBMITTED` y solicita su *receipt* al proveedor Ethereum. Si detecta una discrepancia (por ejemplo, una transaccion minada que no disparo el event listener por un reinicio del servidor), ejecuta una reconciliacion manual sobre el bloque afectado.

Mecanismo de reconciliacion. Si el listener pierde conectividad o el nodo Ethereum se reinicia, el worker puede escanear un rango de bloques recientes, filtrar los logs del contrato y reaplicar los eventos perdidos sobre la base de datos. Este mecanismo garantiza que, incluso ante fallos transitorios, el estado final de PostgreSQL converja con el de la blockchain.

Listing 3: Pseudocodigo del listener y reconciliacion

```

funcion iniciarListener():
  contrato.on('DocumentCreated', (docHash, cid, owner, evento) =>
    baseDatos.actualizarPorHash(docHash, {
      estado: 'SYNCED',
      txHash: evento.log.transactionHash,
      bloque: evento.log.blockNumber
    })
    websocket.emit('documento:actualizado', { docHash })
  fin
fin

```

```
fin funcion

funcion reconciliar(bloqueInicio, bloqueFin):
    eventos = proveedor.obtenerLogs(contrato.filtro, bloqueInicio, bloqueFin)
    para cada ev en eventos:
        si not baseDatos.existeEvento(ev.transactionHash) entonces
            aplicarEvento(ev)
        fin si
    fin para
fin funcion
```

6. Smart Contracts

El sistema utiliza un único contrato inteligente consolidado, **DocumentRegistry**, ubicado en `smart-contracts/contracts/DocumentRegistry.sol`. Este contrato agrupa toda la lógica on-chain: registro de documentos, versionado, firma digital, compartición y control de acceso basado en roles mediante **AccessControl** de OpenZeppelin, además de protección contra reentrancia con **ReentrancyGuard**. La documentación NatSpec y los tests de verificación funcional se encuentran respectivamente en el código fuente del contrato y en `smart-contracts/test/DocumentRegistry.test.js`.

6.1. DocumentRegistry.sol

El contrato **DocumentRegistry** implementa las siguientes responsabilidades: registro de documentos con identificador único, gestión del ciclo de vida de versiones, firma digital por dirección de wallet, asignación y revocación de permisos de lectura y edición, transferencia de propiedad y operaciones de archivado y borrado lógico. La implementación se apoya en las librerías auditadas de OpenZeppelin para control de acceso granular y protección frente a ataques de reentrada.

6.2. Eventos on-chain y verificación funcional

Los diez eventos que emite el contrato constituyen la interfaz de auditoría natural del sistema: **DocumentCreated**, **VersionCreated**, **VersionRestored**, **DocumentSigned**, **DocumentShared**, **PermissionRevoked**, **OwnershipTransferred**, **DocumentArchived**, **DocumentDeleted** y **OperationalVersionChanged**. Cada uno se emite en el momento en que la operación correspondiente es confirmada en bloque, permitiendo reconstruir el historial completo de cada documento.

Se realizó una auditoría funcional del contrato mediante tests alojados en `smart-contracts/test/DocumentRegistry.test.js`. Estos tests verifican la correcta emisión de todos los eventos, el control de acceso basado en roles, la protección contra reentrancia y los casos límite aritméticos. La auditoría cubre el despliegue, la creación de documentos, la gestión de versiones, la firma digital, la compartición y revocación de permisos, la transferencia de propiedad, el archivado y borrado, y las funciones de consulta on-chain. El entorno de ejecución es Hardhat con red local de pruebas.

6.3. Decisión de consolidación en un único contrato

Durante la fase de diseño se evaluó la opción de mantener la lógica distribuida en varios contratos independientes frente a la consolidación en un único **DocumentRegistry**. Se optó por un único contrato por simplicidad, para reducir el coste de gas del despliegue y para evitar problemas de atomicidad entre operaciones que afectan a múltiples aspectos del estado on-chain.

6.4. Decisiones de diseño del contrato consolidado

El contrato consolidado incorpora utilidades de OpenZeppelin y se han tomado decisiones explícitas sobre su arquitectura:

- **No upgradeable (sin proxy).** Se descarto el patron proxy transparente o UUPS para evitar la complejidad adicional de la logica de delegacion y el riesgo de ataques a la funcion de actualizacion. Dado que el proyecto se despliega en un entorno controlado de pruebas, se priorizo la claridad del codigo sobre la mutabilidad futura. Cualquier modificacion requiere un redeploy completo, automatizado mediante el script `reseed-dev.ps1`.
- **ReentrancyGuard.** Las funciones que transfieren propiedad o modifican balances utilizan el modificador `nonReentrant` para mitigar ataques de reentrancia, aunque no haya transferencias de ether directas en el contrato.

7. Arquitectura de cifrado

El sistema implementa una arquitectura de cifrado en tres capas que protege los documentos desde el momento de la autenticacion hasta su registro inmutable en la blockchain. Cada capa aborda un vector de ataque diferente y se complementa con las demas.

7.1. Capa 1: Autenticacion

La primera capa garantiza que unicamente usuarios legitimados accedan al sistema y a las claves de descifrado:

- **Argon2id.** Las contraseñas de los usuarios nunca se almacenan en texto plano; se deriva su hash mediante Argon2id con parametros de memoria, iteraciones y paralelismo configurables para resistir ataques de fuerza bruta en GPU.
- **JWT y sesiones controladas.** Tras la autenticacion exitosa, el servidor emite un token de acceso firmado con una clave secreta de 256 bits, con una validez de 15 minutos. Un token de refresco de 7 dias permite la renovacion de sesiones sin solicitar la contraseña repetidamente. Esta combinacion de token de acceso de corta duracion y refresh token reduce la ventana temporal de uso indebido en caso de compromiso del cliente.
- **Verificacion de email.** El acceso ordinario queda condicionado a que el usuario haya confirmado previamente su direccion de correo electronico, reforzando la vinculacion entre identidad y cuenta operativa.

7.2. Capa 2: Cifrado de contenido

La segunda capa protege el contenido binario de los documentos frente a accesos no autorizados en el servidor o en la red de almacenamiento distribuido:

Se genera una clave simetrica unica por documento y por cada version mediante `crypto.randomBytes(32)`. El archivo se cifra con AES-256-GCM, utilizando un IV aleatorio de 16 bytes y un tag de autenticacion de 16 bytes; el modo GCM proporciona confidencialidad y autenticacion del mensaje sin requerir una funcion de hash adicional.

La clave simetrica se protege a su vez mediante un esquema de cifrado hibrido basado en ECDH sobre la curva P-256. Se cifra con la clave publica del propietario, de modo que solo el poseedor de la clave privada correspondiente puede recuperar la clave AES y descifrar el contenido. Como resultado, el contenido cifrado reside en IPFS mientras que la clave cifrada, el IV y el tag se almacenan en PostgreSQL. Para acceder al documento es necesario comprometer ambos sistemas simultaneamente.

7.3. Capa 3: Blockchain

La tercera capa garantiza la inmutabilidad y trazabilidad de los metadatos críticos sin exponer datos sensibles:

- **Hash de contenido cifrado.** Se calcula el hash SHA-256 del documento ya cifrado. Este hash se registra on-chain, de modo que cualquier alteración del fichero en IPFS queda detectable.
- **Hash de metadatos.** Se calcula un hash estructurado sobre los metadatos del documento (título, autor, fecha) para detectar manipulaciones en la capa relacional.
- **Registro on-chain.** El contrato `DocumentRegistry` almacena el hash del contenido, el CID de IPFS y la dirección del propietario, generando una huella de integridad verificable por terceros sin necesidad de acceso al documento original.
- **No almacenamiento de datos sensibles.** Nunca se escribe en la blockchain información personal, claves ni fragmentos del documento en claro, respetando el derecho al olvido y minimizando el riesgo de exposición pública.

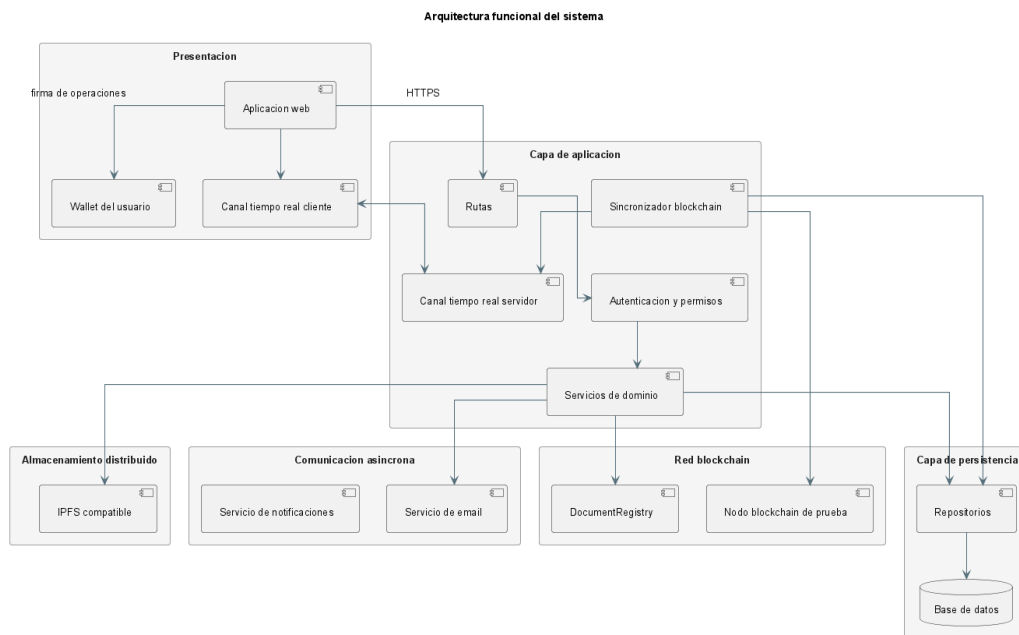


Figura 1: Arquitectura de cifrado de tres capas

La Figura 1 ilustra el flujo completo: el usuario se autentica (Capa 1), el archivo se cifra con una clave única que a su vez se protege con criptografía asimétrica (Capa 2), y finalmente se registra el hash del contenido cifrado en la blockchain (Capa 3).

8. Integración de capas de persistencia

El sistema no podría operar únicamente con blockchain, ni tendría sentido construirlo solo sobre una base de datos relacional si se desea una huella de integridad externa. La arquitectura se apoya en tres capas de persistencia complementarias que conforman el denominado *triángulo de persistencia*.

Capa	Que almacena	Ventajas	Desventajas
PostgreSQL	Usuarios, sesiones, preferencias, metadatos, organización por carpetas y etiquetas, estados de sincronización, notificaciones y proyecciones de consulta.	Permite búsquedas complejas, filtros, actualización frecuente y relaciones con bajo coste operativo.	No garantiza inmutabilidad ni verificación externa.
IPFS	Ficheros documentales y sus versiones, en claro o cifrados según visibilidad.	Proporciona direccionamiento por contenido (CID) y separación entre archivo binario y registro transaccional.	La disponibilidad depende del pinning; sin incentivos económicos, la persistencia no está garantizada por la red pública.
Blockchain	Identificador documental, CID, roles, firmas, cambios de titularidad y estados críticos del documento.	Aporta inmutabilidad, trazabilidad compartida y posibilidad de verificación independiente sin confianza en un tercero.	Coste de gas, latencia elevada, capacidad de almacenamiento limitada y rigidez en las modificaciones.

Cuadro 8: Reparto de responsabilidades entre las tres capas de persistencia

Ese reparto evita dos extremos poco convenientes. Si todo se conservara en PostgreSQL, la aplicación sería más simple pero perdería la dimensión de evidencia externa que justifica el proyecto. Si todo se intentara llevar a Ethereum, el sistema resultaría costoso, rígido y técnicamente desproporcionado para el tipo de operaciones que se ejecutan a diario.

8.1. Funcionamiento de IPFS

IPFS se emplea como capa de almacenamiento distribuido mediante un adaptador configurable en backend. En función del entorno, puede utilizarse un nodo Kubo autoalojado o un proveedor gestionado como Pinata, manteniendo en ambos casos direccionamiento por contenido basado en CID.

Cada archivo subido genera un *Content Identifier* (CID). Ese identificador depende del contenido y no de una ruta lógica decidida por la aplicación. Cuando el documento cambia, aunque solo se altere un byte, se obtiene un CID distinto. Esa propiedad encaja de forma natural con el versionado documental: cada versión activa o restaurada queda asociada a un CID concreto, y la cadena registra precisamente esa referencia inmutable.

El tratamiento del archivo depende de la visibilidad documental. Si el documento es privado, el backend recibe el fichero en bruto, lo cifra con AES-256-GCM, sube a IPFS el resultado cifrado y almacena en la base de datos la clave simétrica cifrada, el vector de inicialización y la etiqueta de autenticación. Si el documento es público, no se aplica ese cifrado del binario y el archivo se conserva en claro.

8.2. Flujo de lectura/escritura coordinado

Las operaciones de lectura y escritura atraviesan las tres capas de forma coordinada:

Escritura. El backend recibe el archivo, lo persiste temporalmente en disco, lo cifra si es necesario, lo sube a IPFS, obtiene el CID, crea el registro en PostgreSQL con estado **PREPARING** y, tras la confirmación blockchain, actualiza el estado a **SYNCED**.

Lectura. El backend consulta PostgreSQL para recuperar los metadatos y la clave cifrada; solicita el contenido al nodo IPFS mediante el CID; descifra la clave simétrica con la clave

privada del usuario; y finalmente descifra el archivo con AES-256-GCM antes de enviarlo al cliente.

9. Smart contract consolidado DocumentRegistry

La implementación actual se concentra en un único contrato, `DocumentRegistry`, situado en `smart-contracts/contracts/DocumentRegistry.sol`. Se trata de un contrato consolidado que integra registro de documentos, versionado, firmas y control de acceso.

Para soportar esa lógica se utilizan utilidades de OpenZeppelin. `AccessControl` permite diferenciar roles administrativos globales; `ReentrancyGuard` limita ataques por reentrada; y `EnumerableSet` mantiene colecciones de documentos y usuarios con operaciones de consulta razonables.

9.1. Estructuras de datos on-chain

El contrato se apoya en tres *structs* principales y en varios *mappings* anidados.

Elemento	Campos principales	Función dentro del sistema
Document	<code>docId</code> , <code>owner</code> , <code>createdAt</code> , <code>updatedAt</code> , <code>currentVersion</code> , <code>latestVersion</code> , <code>isArchived</code> , <code>isDeleted</code>	Representa el estado documental mínimo que debe mantenerse en cadena: titularidad, versión operativa y marcas de estado.
Version	<code>versionNumber</code> , <code>ipfsCid</code> , <code>encryptedKeyHash</code> , <code>createdBy</code> , <code>createdAt</code> , <code>isOperational</code> , <code>restoredFrom</code>	Vincula cada versión con su CID de IPFS y reserva un campo para asociar metadatos criptográficos sin exponer la clave real.
Signature	<code>signer</code> , <code>signature</code> , <code>message</code> , <code>comment</code> , <code>timestamp</code>	Registra la evidencia de firma por dirección Ethereum, preservando el mensaje y el instante en el que se aceptó la firma.
<code>_permissions</code>	<code>docId</code> → <code>address</code> → <code>DocumentRole</code>	Determina el nivel de acceso efectivo de cada cuenta sobre cada documento.

Cuadro 9: Estructuras de datos principales del contrato `DocumentRegistry`

9.2. Operaciones registradas en cadena

Las funciones públicas del contrato reproducen el núcleo funcional del sistema documental. La Tabla 10 resume las operaciones principales, sus parámetros y los eventos asociados.

Funcion	Parametros principales	Eventos emitidos
createDocument	docId, ipfsCid, encryptedKeyHash	DocumentCreated
createVersion	docId, ipfsCid, encryptedKeyHash	VersionCreated
restoreVersion	docId, versionNumber	VersionRestored
setOperationalVersion	docId, versionNumber	OperationalVersionChanged
shareDocument	docId, user, role	DocumentShared
revokePermission	docId, user	PermissionRevoked
signDocument	docId, signature, message	DocumentSigned
transferOwnership	docId, newOwner	OwnershipTransferred
setArchiveStatus	docId, isArchived	DocumentArchived
deleteDocument	docId	DocumentDeleted

Cuadro 10: Funciones publicas y eventos del contrato consolidado

A continuacion se detalla el proposito de cada operacion:

- **createDocument**: crea el documento inicial, registra la primera version y concede al emisor el rol OWNER.
- **createVersion**: anade una nueva version, desactiva la version operativa previa y actualiza el contador.
- **restoreVersion**: genera una version nueva reutilizando el CID y el hash criptografico de una version anterior.
- **setOperationalVersion**: permite senalar otra version ya existente como operativa.
- **shareDocument** y **revokePermission**: gestionan permisos de lectura y edicion para terceros.
- **signDocument**: registra firmas verificables por version y evita duplicados.
- **transferOwnership**: transfiere la titularidad del documento y reajusta los roles del propietario saliente y del entrante.
- **setArchiveStatus** y **deleteDocument**: expresan archivado y borrado logico sin eliminar el rastro historico ya publicado.

9.3. Eventos y restricciones de seguridad

Cada operacion emite eventos que constituyen la interfaz de auditoria natural del sistema. **DocumentCreated** y **VersionCreated** registran altas y versionados; **DocumentShared** y **PermissionRevoked** reflejan cambios de permisos; **DocumentSigned** deja constancia de la firma emitida por una direccion concreta.

Las restricciones de seguridad se expresan mediante modificadores y controles internos. **nonReentrant** reduce la superficie de ataques por reentrada; **notDeleted** bloquea acciones incompatibles con el borrado logico; y **notArchived** impide modificar el estado de documentos archivados. El contrato conserva ademas salvaguardas defensivas adicionales a nivel interno, pero no se documentan como funcionalidades visibles del sistema final.

10. Patron prepare/confirm y sincronizacion

Las operaciones que modifican el estado documental y requieren evidencia on-chain no se ejecutan en un unico paso, porque la firma de la transaccion debe realizarse con la clave privada del usuario, que nunca abandona su wallet.

Fase prepare. Cuando el usuario solicita crear un documento, una version o una firma, el backend realiza todo aquello que no requiere firma blockchain: valida permisos, cifra el binario con AES-256-GCM cuando el documento es privado, sube el resultado a IPFS, genera los identificadores necesarios y crea un registro en PostgreSQL con estado `PREPARING`.

Fase confirm. El frontend usa esos datos para construir la transaccion, la firma con la wallet del usuario y la envia a la red. Una vez que dispone del *transaction hash*, llama al endpoint de confirmacion del backend. Este endpoint recibe el hash y el identificador blockchain, localiza el registro previamente preparado y actualiza su estado a `TX_SUBMITTED`.

Sincronizacion final. Un trabajador periodico (`blockchainSync.ts`) consulta cada cinco minutos los registros en estado `TX_SUBMITTED` y pide el *receipt* de cada transaccion al proveedor Ethereum. Si el *receipt* indica exito (`status === 1`), el registro pasa a `SYNCED`. Si la transaccion fue revertida (`status === 0`), el estado cambia a `FAILED`.

El pseudocodigo detallado de estas fases se presento en la List 2. La separacion de responsabilidades entre fases permite que el frontend se mantenga agnostico respecto a la logica de negocio, mientras que el backend no requiere acceso a las claves privadas de los usuarios.

11. Observabilidad blockchain y sincronizacion de estado

11.1. El patron de observabilidad mediante Event Listener

Una de las decisiones arquitectonicas mas relevantes del sistema es que el backend **no consulta la blockchain para cada operacion de lectura**. En su lugar, mantiene una copia sincronizada del estado on-chain en PostgreSQL, actualizada mediante un mecanismo de observabilidad basado en eventos de Solidity.

El componente encargado de esta sincronizacion es el `Event Listener`, implementado en `backend/src/services/blockchainSync.ts`. Su funcionamiento sigue el siguiente ciclo:

1. **Escucha de eventos.** Al arrancar el backend, el `Event Listener` se suscribe a todos los eventos definidos en el contrato `DocumentRegistry`: `DocumentCreated`, `VersionCreated`, `DocumentShared`, `PermissionRevoked`, `DocumentSigned`, `OwnershipTransferred`, `DocumentArchived`, `DocumentDeleted`, etc.
2. **Procesamiento en tiempo real.** Cuando el minero confirma un bloque que contiene una transaccion del sistema, el nodo Ethereum emite el evento correspondiente. El listener lo captura, extrae sus parametros (identificador de documento, direccion del usuario, nuevo rol, CID, etc.) y actualiza inmediatamente el registro en PostgreSQL.
3. **Sincronizacion periodica.** Ademas del canal de eventos en tiempo real, un trabajador periodico (cada cinco minutos) recorre los registros en estado `TX_SUBMITTED` que aun no han sido confirmados por la via de eventos. Para cada uno, consulta el *receipt* de la transaccion al proveedor Ethereum. Si el *receipt* indica exito (`status === 1`), el registro pasa a `SYNCED`. Si fue revertida (`status === 0`), se marca como `FAILED`.
4. **Tolerancia a fallos.** Si el listener pierde la conexion con el nodo Ethereum o el backend se reinicia, el trabajador periodico garantiza que los eventos emitidos durante la ventana de desconexion se recuperen consultando los bloques desde el ultimo bloque procesado.

Este diseño ofrece dos ventajas fundamentales. Primero, las lecturas de metadatos, listados de documentos y consultas de permisos se resuelven contra PostgreSQL, con la latencia y la flexibilidad de consulta de una base de datos relacional. Segundo, la blockchain funciona como fuente de verdad auditada: cualquier discrepancia entre el estado en PostgreSQL y el estado on-chain puede detectarse cotejando los eventos históricos.

11.2. Flujo completo de una operación documental

Para ilustrar la coordinación entre las tres capas (frontend, backend y blockchain), se describe a continuación el flujo de creación de un documento privado con firma, que constituye la operación más representativa del sistema:

1. **Subida del archivo.** El usuario selecciona un archivo y metadatos en la interfaz. El frontend envía el binario al backend mediante una petición HTTP multipart.
2. **Cifrado y almacenamiento.** El backend genera una clave simétrica AES-256-GCM aleatoria, cifra el archivo con ella, y sube el resultado cifrado a IPFS. Obtiene el CID.
3. **Preparación de la transacción.** El backend crea un registro en PostgreSQL con estado `PREPARING`, cifra la clave simétrica con la clave pública del usuario (ECDH P-256), y devuelve al frontend los datos necesarios para construir la transacción blockchain.
4. **Firma del usuario.** El frontend presenta los datos a la wallet del usuario (Metamask o WalletConnect). El usuario revisa y firma la transacción.
5. **Confirmación.** El frontend envía el *transaction hash* al backend. El backend marca el registro como `TX_SUBMITTED`.
6. **Sincronización on-chain.** El `Event Listener` detecta el evento `DocumentCreated` en el siguiente bloque. El backend actualiza el estado del documento a `SYNCED` y la interfaz del usuario refleja el cambio mediante WebSocket.

Este flujo garantiza tres propiedades críticas: el archivo nunca viaja sin cifrar fuera del control del backend, la clave privada del usuario nunca abandona su wallet, y la evidencia de la operación queda registrada en la blockchain de forma inmutable.

12. Gestión de contraseñas, frases de recuperación y claves

12.1. Derivación de claves con Argon2

El sistema utiliza el algoritmo **Argon2id** (ganador del Password Hashing Competition) para derivar claves a partir de la contraseña del usuario. Argon2id combina las variantes Argon2i (resistente a *side-channel attacks*) y Argon2d (resistente a ataques GPU), ofreciendo protección equilibrada frente a ambos vectores.

Cuando el usuario se registra, el backend genera una sal aleatoria de 32 bytes y aplica Argon2id con los siguientes parámetros: `memoryCost` = 65536 KB, `timeCost` = 3 iteraciones, `parallelism` = 4 hilos. El hash resultante se almacena en la tabla `User` de PostgreSQL. La contraseña en texto plano nunca se conserva en disco ni en memoria más allá del tiempo necesario para completar el proceso de derivación.

12.2. Cifrado de claves privadas con frase de recuperacion

La clave privada de cada usuario (utilizada para el intercambio ECDH en la capa de cifrado de documentos) se protege mediante un mecanismo adicional. La clave se cifra con AES-256-GCM utilizando una clave derivada de la frase de recuperacion del usuario. Esta frase de recuperacion se genera durante el registro y se muestra al usuario una unica vez, instandole a guardarla de forma segura.

Esta separacion entre la contraseña de acceso (que puede cambiarse) y la frase de recuperacion (que es inmutable y custodia la clave privada) sigue el principio de minimo privilegio. Si un atacante compromete la contraseña del usuario, aun necesitaria la frase de recuperacion para descifrar los documentos, ya que la clave privada necesaria para el intercambio ECDH permanece cifrada con esa segunda capa de proteccion.

13. Despliegue tecnico con Docker

La materializacion operativa de esta arquitectura se apoya en Docker Compose. El motivo principal no es unicamente la comodidad de arranque, sino la necesidad de conservar una topologia estable entre servicios que dependen unos de otros: PostgreSQL para persistencia relacional, Hardhat como red EVM local, backend Node.js para la logica de aplicacion, frontend servido por Nginx, Postfix para correo transaccional y un nodo Kubo para almacenamiento distribuido.

13.1. Arquitectura de contenedores

La Figura 2 muestra la arquitectura de contenedores y sus interdependencias. Cada servicio se ejecuta en su propio contenedor aislado, comunicandose mediante una red virtual interna gestionada por Docker Compose.

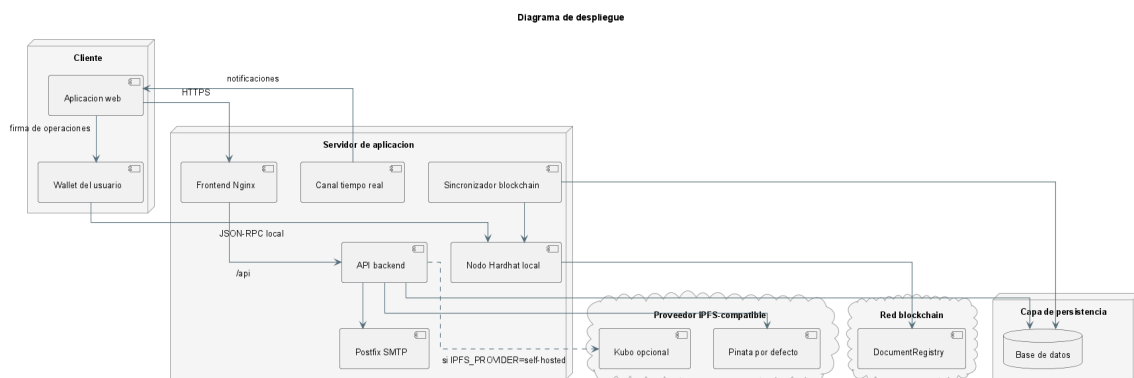


Figura 2: Arquitectura de despliegue con contenedores Docker

13.2. Servicios del docker-compose

El fichero `docker-compose.yml` define los siguientes servicios:

- **postgres:** Base de datos relacional PostgreSQL 16. Almacena metadatos, usuarios, sesiones, estados de sincronizacion y claves cifradas. Expone el puerto 5432 internamente.
- **ipfs-node:** Nodo Kubo oficial de IPFS. Gestiona el almacenamiento distribuido de archivos, el pinning local y la API HTTP en el puerto 5001. Su volumen persistente garantiza que los CIDs no se pierdan ante reinicios.

- **hardhat:** Red Ethereum local para desarrollo y pruebas. Se ejecuta con la configuracion por defecto de Hardhat (cuentas pre-fundidas, minado automatico). Expone el RPC en el puerto 8545.
- **backend:** Servicio Node.js que encapsula la API REST. Depende de **postgres** e **ipfs-node**. Se reconstruye automaticamente ante cambios en el codigo fuente durante el desarrollo.
- **frontend:** Imagen de produccion basada en Nginx. Sirve los assets estaticos generados por Vite y redirige las rutas de la SPA al fichero **index.html**.
- **nginx:** Proxy inverso que expone los puertos 80 y 443 hacia el exterior. Redirige las peticiones a **/api** al backend y el resto al contenedor del frontend. Termina las conexiones SSL/TLS cuando se configuran certificados.
- **postfix:** Servidor de correo para el envio de notificaciones transaccionales (verificacion de cuenta, recuperacion de acceso) sin depender de servicios SMTP externos.

13.3. Volúmenes, redes y variables de entorno

Volúmenes. Se definen volúmenes Docker persistentes para **postgres-data** (ficheros de base de datos), **ipfs-data** (repositorio de bloques y configuracion del nodo Kubo) y **backend-logs** (ficheros de trazas de Winston). Esta separacion garantiza que los datos sobrevivan a la recreacion de contenedores.

Redes. Todos los servicios se conectan a una red bridge interna denominada **documentchain-network**. El aislamiento de red impide que servicios externos accedan directamente a PostgreSQL o al API de IPFS sin pasar por Nginx y el backend.

Variables de entorno. La configuracion sensible (claves JWT, credenciales de base de datos, URL del nodo Ethereum) se inyecta mediante un fichero **.env** que no se versiona en el repositorio. Durante el despliegue mediante GitHub Actions, estas variables se configuran como secretos del repositorio.

13.4. Integracion continua (CI/CD)

El proyecto incorpora un *workflow* de *GitHub Actions* orientado a *runners* autoalojados en Linux. Cuando llega un *push* a la rama principal, el pipeline ejecuta las siguientes fases:

1. **Checkout y dependencias.** Clona el repositorio e instala las dependencias de frontend, backend y smart contracts.
2. **Tests.** Ejecuta los tests unitarios con Vitest (frontend), Jest (backend) y Hardhat (smart contracts). Si algun test falla, el pipeline se interrumpe.
3. **Build.** Compila el frontend para produccion, transpila el backend y genera los artefactos de despliegue.
4. **Despliegue.** El *runner* autoalojado actualiza el codigo en el servidor, reconstruye las imagenes Docker necesarias, aplica las migraciones Prisma pendientes y reinicia los servicios mediante Docker Compose.

Una decision arquitectonica explicita del proyecto es no utilizar un patron de proxy upgradeable para el contrato inteligente. Aunque los proxies permiten actualizar la logica sin cambiar su direccion, su implementacion anade complejidad adicional. En este trabajo se ha priorizado la claridad del codigo y la reproducibilidad del entorno de pruebas. Como consecuencia, cualquier modificacion en **DocumentRegistry** exige un redeploy completo. El script **reseed-dev.ps1** automatiza precisamente ese ciclo.

14. Glosario tecnico

ABI Application Binary Interface, interfaz que define como interactuar con un smart contract desde codigo externo.

AES-256-GCM Advanced Encryption Standard en modo Galois/Counter Mode con clave de 256 bits. Proporciona confidencialidad y autenticacion simultaneas.

Argon2 Algoritmo de derivacion de claves ganador del Password Hashing Competition, resistente a ataques de paralelizacion en GPU.

CID Content Identifier, identificador unico de contenido en IPFS basado en hash criptografico.

ECDH Elliptic Curve Diffie-Hellman, protocolo de intercambio de claves sobre curvas elipticas que permite acordar un secreto compartido de forma segura.

EVM Ethereum Virtual Machine, entorno de ejecucion de smart contracts en redes compatibles con Ethereum.

Event Listener Servicio del backend que escucha eventos emitidos por el smart contract para sincronizar estados.

Gas Unidad de medida del coste computacional de operaciones en la EVM; se paga en la moneda nativa de la red.

Hardhat Entorno de desarrollo, testing y despliegue de smart contracts en la EVM.

IPFS InterPlanetary File System, protocolo y red de almacenamiento distribuido por contenido.

JWT JSON Web Token, estandar compacto y autocontenido para la transmision segura de informacion entre partes.

Kubo Implementacion oficial de nodo IPFS desarrollada en Go; se usa en el proyecto para almacenamiento autoalojado.

Mapping Estructura de datos tipo diccionario en Solidity que relaciona claves con valores.

Metamask Billetera digital de navegador que permite gestionar claves privadas e interactuar con aplicaciones descentralizadas.

NatSpec formato de documentacion embebida en comentarios de Solidity para generar especificaciones legibles por maquina.

Pinning Accion de mantener contenido permanentemente disponible en un nodo IPFS, evitando que sea eliminado por el recolector de basura.

Prisma ORM de siguiente generacion para Node.js y TypeScript que proporciona tipado seguro y migraciones.

Reentrancia Vulnerabilidad por la que una funcion externa puede ser llamada repetidamente antes de que termine la ejecucion original; mitigada con **ReentrancyGuard**.

Sepolia Red de pruebas publica de Ethereum utilizada para validar contratos antes de un despliegue en mainnet.

shadcn/ui Coleccion de componentes React reutilizables contruidos sobre Radix UI y Tailwind CSS.

TxHash Transaction Hash, identificador unico de una transaccion confirmada en blockchain.

UML Unified Modeling Language, lenguaje grafico estandar para la especificacion, visualizacion y documentacion de sistemas software.

Vite Herramienta de construccion frontend que utiliza ES Modules nativos para un desarrollo rapido.

WalletConnect Protocolo abierto que permite conectar wallets moviles con aplicaciones descentralizadas mediante codigos QR.

15. Conclusiones del Anexo IV

El presente anexo ha documentado la totalidad del codigo fuente del sistema DocumentChain, incluyendo las tecnologias empleadas, la estructura modular del proyecto, los metodos criticos de cifrado y sincronizacion, los smart contracts principales, la arquitectura de cifrado de tres capas, el reparto de responsabilidades entre PostgreSQL, IPFS y blockchain, el patron prepare/confirm y el despliegue mediante Docker.

Se ha prestado especial atencion a la reproducibilidad del entorno, la trazabilidad de las decisiones de diseno y la separacion de responsabilidades entre capas. La documentacion tecnica aqui presentada constituye la referencia definitiva para cualquier desarrollador que deba mantener o extender el sistema, asi como para auditores que requieran comprender los flujos de cifrado, sincronizacion y control de acceso sin necesidad de reconstruir la logica a partir del codigo fuente exclusivamente.